

1 Synchronisation mit Semaphoren (30)

Verwenden Sie Semaphore zur Lösung des folgenden Synchronisationsproblems zwischen den zyklischen Prozessen *A*, *B*, *C*, *D* und *E*. Prozess *A* schreibt immer wieder Daten auf ein Shared Memory, in dem Platz für einen Datensatz ist. *B*, *C* und *D* lesen jeden von *A* geschriebenen Datensatz vom Shared Memory und verarbeiten diesen. Nach der Fertigstellung der Verarbeitung durch *B*, *C* und *D* wird im Prozess *E* die Funktion *finish()* ausgeführt. Beachten Sie folgendes:

- Jeder Datensatz, der von *A* mittels *write_ShM()* auf das Shared Memory geschrieben wird, muss von *B*, *C* und *D* genau einmal mittels *read_ShM()* gelesen werden.
- *B*, *C* und *D* müssen einzelne Datensätze parallel verarbeiten können.
- Nachdem *B*, *C* und *D* vom Shared Memory gelesen haben, verwenden sie die Funktion *use_data()*, um die vom Shared Memory gelesenen Daten zu verarbeiten.
- Wenn die drei Prozesse mit der Verarbeitung der Daten fertig sind, ruft *E* die Funktion *finish()* auf. Danach beginnt die nächste Verarbeitungsrunde, d.h. *A* schreibt wieder auf das Shared Memory.

Ergänzen Sie die Codestücke für die Prozesse *A*, *B*, *C*, *D* und *E* und geben Sie passende Initialisierungen an.

Initialisierungen

```
init(A, 1);  
init(B, 0);  init(Br, 0);  
init(C, 0);  init(Cr, 0);  
init(D, 0);  init(Dr, 0);
```

```
/** Prozess A **/
```

```
for(;;) {
```

```
wait(A);
```

```
writeShm(data);
```

```
signal(B);
```

```
}
```

```
/** Prozess B **/
```

```
for(;;) {
```

```
wait(B);
```

```
readShm(&data);
```

```
signal(C);
```

```
signal(Br);
```

```
signal(Br);
```

```
wait(Er);
```

```
wait(Dr);
```

```
use_data(data);
```

```
signal(E);
```

```
}
```

```
/** Prozess C **/
```

```
for(;;) {
```

```
wait(C);
```

```
readShm(&data);
```

```
signal(D);
```

```
signal(Cr);
```

```
signal(Cr);
```

```
wait(Dr);
```

```
wait(Dr);
```

```
use_data(data);
```

```
signal(E);
```

```
}
```

```
/** Prozess D **/
```

```
for(;;) {
```

```
wait(D);
```

```
readShm(&data);
```

```
signal(Dr);
```

```
signal(Dr);
```

```
wait(Br);
```

```
wait(Cr);
```

```
use_data(data);
```

```
signal(E);
```

```
}
```

```
/** Prozess E **/
```

```
for(;;) {
```

```
wait(E);
```

```
wait(E);
```

```
wait(E);
```

```
finish();
```

```
signal(A);
```

```
}
```


2 Memory Management (30)

Für einen Arbeitsspeicher mit Paging ist eine Seitenzugriffsfolge gegeben. Für diese Zugriffsfolge sollen das Working Set sowie die Belegung der Pages nach jedem Speicherzugriff bei Verwendung unterschiedlicher Seiten-Ersetzungsstrategien beschrieben werden.

Working Set: Geben Sie in jeder Spalte der Tabelle das Working Set $W(D, t)$ mit $D = 5$ nach jedem in der Kopfzeile gegebenen Seitenzugriff an.

$W(5, t)$	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
	A	B A	C B A	D C B A	B D C A	E B D C	A E B D C	F A E B D	D F A E B	E D F A	B E D F A	D B E F	B D E	F B D E	E F B D	D E F B	F D E B	C F D E B

Page Replacement: Geben Sie nun in den Spalten der Tabellen die Speicherinhalte für jeden Frame nach jedem in der Kopfzeile angegebenen Seitenzugriff bei Verwendung der Replacement Strategien LRU bzw. FIFO an. Kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Die anfängliche Belegung des Arbeitsspeichers ist in der Tabelle eingetragen. Die Seiten wurden in der Reihenfolge *E, A, B, C* in den Speicher geladen.

LRU	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
0	E	E	E	D	D	D	D	F	F	F	F	F	F	F	F	F	F	F
1	A	A	A	A	A	E	E	E	E	E	E	E	E	E	E	E	E	E
2	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D	D	D	D
3	C	C	C	C	C	C	A	A	A	A	B	B	B	B	B	B	B	C
PF	-	-	-	X	-	X	X	X	X	-	X	-	-	-	-	-	-	X

FIFO	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
0	E	E	E	D	D	D	D	D	D	D	B	B	B	B	B	B	B	B
1	A	A	A	A	A	E	E	E	E	E	D	D	D	D	D	D	D	D
2	B	B	B	B	B	B	A	A	A	A	A	A	A	A	E	E	E	E
3	C	C	C	C	C	C	C	F	F	F	F	F	F	F	F	F	F	C
PF	-	-	-	X	-	X	X	X	-	-	X	X	-	-	X	-	-	X